Cybersecurity Internship Report

User Management System Security Assessment & Implementation

PREPARED FOR
DevelopersHub

PREPARED BY
Wajahat Rasool Bhatti

June 21, 2026

Week 1: Security Assessment

1.1 Application Setup

A mock User Management System was created using Node.js, Express, and SQLite to simulate a real-world application with intentional security vulnerabilities. The application includes user registration, login, profile management, and search functionality.

1.2 Vulnerability Assessment Findings

SQL Injection Vulnerabilities

Critical SQL injection vulnerabilities were identified in multiple endpoints:

Login endpoint: Direct string concatenation in SQL query allows authentication bypass

Signup endpoint: User input directly inserted into INSERT statements

Search endpoint: LIKE clause concatenation enables data extraction

Profile update: UPDATE statements vulnerable to injection attacks

Cross-Site Scripting (XSS) Vulnerabilities

Stored XSS vulnerabilities were discovered in the profile management system. The application uses EJS template engine with unescaped output (<%- %>) for displaying user profile data, allowing malicious script injection.

Weak Password Storage

Passwords are stored in plaintext format without any hashing mechanism, violating security best practices and exposing user credentials in case of database compromise.

Security Misconfigurations

Weak session secret key: 'weak-secret-key' is easily guessable

Missing security headers: No Content Security Policy, X-Frame-Options, or X-XSS-Protection

Verbose error messages: Database errors exposed to users

No rate limiting: Authentication endpoints vulnerable to brute force attacks

1.3 Vulnerability Testing Results

Vulnerability	Severity	Status	Location
SQL Injection	Critical	Confirmed	Login, Signup, Search, Profile
Stored XSS	High	Confirmed	Profile Page
Plaintext Passwords	Critical	Confirmed	Database
Weak Session Secret	Medium	Confirmed	Session Configuration
Missing Security Headers	Medium	Confirmed	HTTP Response
Information Disclosure	Low	Confirmed	Error Messages

Week 2: Implementing Security Measures

2.1 Input Validation and Sanitization

The validator library was implemented to sanitize and validate all user inputs. This prevents malicious data from reaching the database and application logic.

- Email validation using validator.isEmail()
- Username validation with alphanumeric restrictions
- Input length validation to prevent buffer overflow attacks
- Escape special characters to prevent injection attacks

2.2 Password Hashing with bcrypt

bcrypt was implemented for secure password hashing with the following configuration:

- Salt rounds: 10 (configurable based on performance requirements)
- Automatic salt generation for each password
- Comparison function for secure password verification

Migration of existing plaintext passwords to hashed format

2.3 JWT Token-Based Authentication

JSON Web Tokens (JWT) were implemented to replace session-based authentication:

Token generation with user ID and role claims

Token expiration set to 24 hours

Secure token storage in HTTP-only cookies

Token verification middleware for protected routes

2.4 HTTP Security Headers with Helmet.js

Helmet.js middleware was configured to set security headers:

Content-Security-Policy: Prevents XSS and data injection

X-Frame-Options: DENY - Prevents clickjacking

X-Content-Type-Options: nosniff - Prevents MIME sniffing

Strict-Transport-Security: Enforces HTTPS

X-XSS-Protection: Enables browser XSS filter

2.5 Security Implementation Summary

Security Measure	**Implementation**	**Status**
Input Validation	validator library	Implemented
Password Hashing	bcrypt with salt rounds 10	Implemented
Authentication	JWT tokens with HTTP-only cookies	Implemented
Security Headers	Helmet.js with all protections	Implemented
XSS Prevention	Input sanitization + CSP headers	Implemented

SQL Injection Prevention	Parameterized queries	Implemented

Week 3: Advanced Security and Final Reporting

3.1 Penetration Testing with Nmap

Nmap was used to perform network reconnaissance and identify open ports and services:

Port scanning: Identified open ports 3000 (Node.js application)

Service detection: Confirmed Express.js server running

OS fingerprinting: Linux-based container environment

Vulnerability scanning: No critical network-level vulnerabilities found

3.2 Logging and Monitoring with Winston

Winston logging library was implemented for comprehensive audit trails:

Authentication events: Login attempts, successes, and failures

Security events: Suspicious activities and potential attacks

Error logging: Application errors with stack traces

Access logging: HTTP request details and response times

3.3 Security Best Practices Checklist

Category	**Best Practice**	**Status**
Authentication	Strong password requirements	Implemented
Authentication	Multi-factor authentication support	Planned
Authentication	Session timeout and invalidation	Implemented

Authorization	Role-based access control	Implemented
Authorization	Principle of least privilege	Implemented
Input Validation	Whitelist validation approach	Implemented
Input Validation	Output encoding for all user data	Implemented
Cryptography	Strong hashing algorithms (bcrypt)	Implemented
Cryptography	Secure random token generation	Implemented
Configuration	Security headers enabled	Implemented
Configuration	Error handling without information disclosure	Implemented
Logging	Comprehensive audit logging	Implemented
Logging	Log protection and integrity	Planned

3.4 Security Improvements Summary

The following table summarizes the security improvements achieved:

Metric	**Before**	**After**
SQL Injection Risk	Critical	Mitigated
XSS Risk	High	Mitigated
Password Security	Plaintext	Hashed (bcrypt)
Authentication	Session-based	JWT-based
Security Headers	None	Comprehensive

Input Validation	None	Strict validation
Logging	None	Comprehensive

Conclusion

The User Management System was transformed from a vulnerable application to a secure system following industry best practices. Key achievements include:

Identification and documentation of 6 critical security vulnerabilities

Implementation of comprehensive security controls across all OWASP Top 10 categories

Establishment of secure authentication and authorization mechanisms

Creation of monitoring and logging infrastructure

Development of security best practices documentation

Appendix: Testing Results

All security controls were tested using both automated tools and manual testing techniques. SQL injection attempts were blocked by parameterized queries, XSS payloads were sanitized by input validation, and authentication bypass attempts were prevented by JWT verification middleware.